Pip: giving a free-tier AI chat controlled hands on a student's own machine

Project report — iWISE Regional Final 2026

Category	iTECH — Software / Programming
Competition	iWISE Regional Final, Yogyakarta, 29 October – 1 November 2026
Participants	Haidar Ali Fawwaz Nasirodin · Faiq Abiyyu Asyam · Avandreda Giano Bestafa
Country	Indonesia
Supervising teacher	Muhammad Hasan, Ph.D.
Repository	github.com/haidaralif-24/iWISE

Abstract

Students use AI chat daily, yet giving it their own documents is still manual. They copy text out of PDFs, Word files and slides, and the broken formatting burns free-tier limits. The AI cannot see their files, so every question needs pasted context. Autonomous agents would remove that friction, but they require paid API keys or capable hardware, which many students do not have.

Pip is a small always-on-top desktop window, built with Tauri and TypeScript, that opens the AI chat service a user already has beside a set of local tools. A local Model Context Protocol server gives the AI controlled access to the user's files, guarded by a folder allowlist with path canonicalisation, native confirmation prompts before any write, authenticated connections, output size limits and an append-only audit log, while the chat webview is deliberately granted no application permissions. A local pipeline built on Microsoft's MarkItDown converts PDF, DOCX, PPTX and XLSX files and lets the user preview and edit the Markdown.

Pip needs no API key and no local model. The full path from document to tool call was verified on a free Claude account through a public tunnel, and adversarial testing closed path traversal and unauthenticated access while exposing an approval bypass that a green test suite had missed. A separate measurement showed that conversion alone does not reduce token cost; page selection does, cutting a six-page PDF from 2,067 tokens to 686.

---

1. Introduction

A student's AI chat can answer almost anything except the questions that need their own files. It cannot see the essay being written, the notes from last week, or the worksheet in front of them. Everything has to be pasted in by hand, and pasting is where the work goes: PDF line breaks become hard wraps, tables become noise, and part of a limited free tier is spent reading layout instead of content.

The obvious fix is an agent: software that gives the model the ability to read files itself. That fix exists, and it is out of reach. Agent tooling assumes either an API key billed per token or a local model large enough to run on a capable machine. A seventeen-year-old with a school laptop and a

free account has neither. The capability gap is therefore not technical in origin; it is a pricing and hardware gap, and it falls on exactly the users who would benefit most.

Pip is an attempt to close that gap without asking anyone to pay. The design decision that shapes everything else is this: **do not bring the model to the files, bring the files to the model on request.** The model is given a *handle* — a path it may ask about — rather than a *copy*. A small server on the user's own machine answers those requests, and a permission layer decides which requests are allowed.

This has a consequence worth stating early, because it is the project's central claim: the file never moves. Only the text the model explicitly asks for enters the conversation. That is a different proposition from uploading a document, where a copy is stored somewhere the user does not control. It is also the reason page-level reading is possible at all, and therefore the reason any token saving exists.

1.1 What was built

A macOS desktop application in three layers, listed in order of importance rather than order of construction:

- **The tool layer.** A local Model Context Protocol server exposing seven tools — list a folder, read a document, search notes, write a note, render a digest, render a quiz — behind a folder allowlist, an approval prompt for anything that writes, and an append-only audit log. This is the part that gives the chat hands, and the part the project is about.
- **The window.** A Tauri application: a Rust backend, a TypeScript frontend rendered in the system webview, always-on-top, summoned by a global hotkey. It hosts the user's existing chat service in a separate webview that is deliberately granted no application permissions, and carries a Pomodoro timer and quick notes so that it is pleasant enough to leave open.
- **The document pipeline.** A Python sidecar wrapping Microsoft's MarkItDown, which converts PDF, DOCX, PPTX and XLSX to Markdown so the user can read it rendered, edit the source and select pages before the prompt is
  assembled. It is one of the seven tools, and the smallest of the three layers in what it contributes.

---

2. Research question and aims

2.1 Research question

> Can a free-tier cloud AI chat service be given controlled, auditable access to a student's local files using only local components, and how much of the resulting risk surface can a user-controlled permission layer close?

The question has two halves, and both require evidence. A working demonstration answers the first. Only adversarial testing answers the second.

2.2 Secondary question

> Which failures in such a system are detectable by automated testing, and which survive a passing test suite?

This question was not part of the original design. It emerged during the build, when a defect was found by reading code that every test had passed. It turned out to be the most productive line of enquiry in the project, and section 7 is devoted to it.

2.3 Aims

1. Build a desktop application that reuses the AI account a student already has, and gives that AI access to the student's own files under a boundary the student controls.
2. Evaluate the application on three axes: **function** (does it run), **cost** (what it costs the student), and **safety** (what it stops).
3. Report the results honestly, including the ones that contradicted the design.

The aims deliberately exclude "make AI cheaper" and "save tokens". Both were considered and both are too narrow: the first is a vendor's problem, and the second turned out to be a single feature rather than a thesis.

---

3. Background

3.1 Model Context Protocol

MCP is an open standard for exposing tools to an AI client over a transport the client already speaks. Its significance for this project is specific: it is what allows a cloud-hosted chat service to invoke a process on the user's own machine without a bespoke integration for each vendor. Before MCP, "give the model access to my files" meant writing a plugin for one product; with MCP, it means running a server that any compliant client can reach.

Pip implements the server side. It speaks Streamable HTTP on `127.0.0.1:8787`, is stateless, and requires a bearer token on every request.

3.2 MarkItDown

Microsoft's MarkItDown converts a range of document formats into Markdown while preserving heading structure. It was chosen over writing a converter because conversion is a solved problem and the interesting problem in this project is the permission boundary. MarkItDown is invoked through `convert_local()` only — never `convert()` or `convert_url()` — so the sidecar cannot be steered at a remote URI.

3.3 Tauri

Tauri renders a web frontend inside the operating system's own webview, with a Rust backend. It was chosen over Electron for two reasons. The first is binary size. The second matters more: in Tauri, calling out to the operating system is an explicit, reviewable command, whereas Electron's Node integration makes the same capability ambient. For a project whose subject is a permission boundary, an ambient capability is the wrong foundation.

3.4 The gap in existing tools

AI desktop clients already give an assistant access to local files. Two properties make them unsuitable for this user. The boundary is set by the vendor rather than the user, and the account is normally a paid tier. Neither is unreasonable as a product decision; both exclude the student this project is for.

4. Hypotheses

	Statement	Outcome
H1 — Capability	A free-tier cloud chat account can reach a local MCP tool server and execute a tool call, with no API key, no local model and no paid plan.	**Supported**
H2 — Boundary	A user-controlled permission layer — folder allowlist, path canonicalisation, approval before writes, token authentication, audit log — closes the majority of the risk surface of exposing local tools to a remote model.	**Partially supported**
H3 — Efficiency	Converting a document to Markdown reduces the token cost of a prompt.	**Rejected** — conversion made prompts 12% and 4.5% *larger* on the two documents measured
H3 revised	Reading fewer pages reduces token cost, and page-level reading is only possible because the file stays local.	**Supported**

H3 was written before the first measurement and is reported as rejected. The rejection is a result, not a failure: it is the finding most likely to save another student a week, and it is what produced the measurement method used in section 6.3.

The engineering goal was stated separately from H1 so that a working demonstration could not be mistaken for evidence: a document must travel from drag-and-drop to an assembled prompt, and a tool call must reach an approved write, in one continuous run.

5. Methodology

5.1 Build strategy

The application was built in three tiers over seven days, so that each tier would be a complete project if time ran out:

1. **Tier 1** — the window: always-on-top, global hotkey, Pomodoro timer, quick notes.
2. **Tier 2** — the document pipeline: drag-and-drop, local conversion, rendered Markdown preview with an editable source, page
   selection, token counter.
3. **Tier 3** — the tool layer: seven MCP tools, folder allowlist, approval prompt, audit log.

The riskiest unknown — whether a free account could reach a local server at all — was tested on day two, before Tier 2 was begun. A project that fails H1 is a different project, and it was worth knowing that early rather than late.

5.2 Evaluation, axis 1 — Function

The chain was decomposed into seven links, each recorded as pass, fail or untested. A link counts as *pass* only when observed in the running application, never when it merely compiles or passes a unit test:

1. Drag a file onto the Docs tab
2. The application spawns the converter process
3. MarkItDown returns a JSON envelope
4. The preview renders
5. Page selection updates the token counter
6. The AI calls a tool and receives a result
7. A write is blocked, approved, executed and logged

5.3 Evaluation, axis 2 — Safety

Nine adversarial cases, each an attempt to do something the design says is impossible. A case passes only if the attempt is refused **and** the refusal is logged. Crucially, each case asserts the *specific reason* for refusal, not merely that the operation failed — a distinction that turned out to matter (section 7.4).

#	Attempt	Expected
1	Relative traversal, `../../../../etc/passwd`	refused by containment
2	Absolute path outside the allowlist, `/etc/passwd`	refused by containment
3	No `Authorization` header with an empty configured token	refused
4	Unauthenticated request through the public tunnel	refused
5	Read a folder never added to the allowlist	refused
6	Write with no approval decision	held pending
7	Symlink inside an allowed folder pointing outside it	refused by containment
8	A note the model wrote, read back as an instruction	fenced as data
9	Folder swapped between validation and conversion	**known open**

5.4 Evaluation, axis 3 — Cost and efficiency

Marginal cost to the student was measured as three components: subscription delta (the account already exists — is a higher tier required?), tunnel tier, and hardware (an existing laptop or a purchase?). The comparison arm is the same documents sent through a paid API at list price.

Token efficiency is reported as a **sub-metric**, not as the project's thesis. Token counts are produced by a shared estimator — approximately one token per four characters, plus half a token per non-ASCII character — used identically by the application and the server so the figures are comparable. It is not a tokeniser count, and section 9 says so.

---

6. Results

6.1 Function

#	Link	Result
1	Drag a file onto the Docs tab	Pass
2	Spawn the converter process	Pass
3	MarkItDown returns a JSON envelope	Pass
4	The preview renders	Pass
5	Page selection updates the counter	Pass
6	The AI calls a tool and receives a result	Pass
7	A write is blocked, approved, executed and logged	**Pending**

Six of seven links were observed. The seventh is implemented and unit-tested on both halves — the server raises the request, the application answers it — but the two halves have not been watched communicating in a single run. It is recorded as pending rather than passed, because “the halves are tested” is not the same claim as “the whole works”.

6.2 Safety

Eight of nine cases pass. The ninth is open by design.

Case	Outcome
Relative traversal	Refused; refusal names containment
Absolute path outside the allowlist	Refused
Empty token with no credentials	Refused (after being found broken — section 7.1)
Unauthenticated request through the tunnel	HTTP 401
Ungranted folder	Refused
Write without approval	Held pending, not executed
Symlink escaping the allowlist	Refused; refusal names containment
Note read back as an instruction	Fenced as untrusted data (after being found unfenced — section 7.2)
Folder swapped between check and use	**Open**

The open case is a time-of-check-to-time-of-use race: a path is validated, and then used, and the filesystem can change in between. Closing it requires holding a file descriptor or running the conversion inside an OS-level sandbox. Neither is achievable in JavaScript, and both are out of scope for a seven-day build. It is listed rather than hidden because a judge who finds it unlisted learns something about the team; a judge who finds it listed learns something about the project.

6.3 Cost

Component	Cost to the student
Subscription delta	Rp 0 — the existing free account is used unchanged
Tunnel tier	Rp 0 — free tier is sufficient for a demonstration
Hardware	Rp 0 — an existing laptop
API key	none required

The comparison arm is a paid API at list price, which for a student doing coursework runs to a monthly bill that the free account does not. The margin is therefore the whole cost, not a percentage of it.

6.4 One tool, measured: the document converter

The converter is the only part of Pip that can be measured on its own, so it was. It is worth being clear about what it is: a two-hundred-line wrapper around Microsoft's MarkItDown, one of seven tools, included because reading a PDF should not require a separate application. It is not the contribution.

Condition	Tokens	Against pasting
Paste the document by hand	1,844	—
Convert to Markdown, send everything	2,067	**+12% (worse)**
Convert, send pages 2–3 only	**686**	**–63%**

Conversion on its own makes the prompt *larger*, by 12% on this document and 4.5% on the DOCX below. The structure MarkItDown adds — headings, list markers, table pipes — costs characters, and characters are tokens.

What wins is selection, and selection is a property of the architecture rather than of the converter: a model holding a path can ask for a page range. Sending two pages of six costs 686 tokens against 1,844 for pasting the whole document, a 63% reduction; against sending the converted whole document it is a 67% reduction. Six of the 686 tokens are the `[p.2]` and `[p.3]` markers that let the model cite a page rather than "the document".

A second measurement, on a one-page DOCX activity sheet, produced 552 tokens of raw text against 577 of Markdown — again larger, and again no saving from conversion alone.

6.5 The engineering goal

Not achieved as stated. The chain was verified link by link but never in one continuous run before this report was written. The distinction is recorded here rather than smoothed over.

---

7. Errors found, and what they taught

This section answers the secondary research question. Four defects were found during the build, and the pattern in how they were found is more useful than any of them individually.

7.1 An empty token authorised every request

The server's token check compared a configured token against the request header. When the configured token was an empty string, it compared equal to a *missing* header, so a request with no credentials returned HTTP 200 and a tool result.

Found by: adversarial testing, case 3. **Fixed by:** rejecting empty credentials before comparison, and refusing to start with a token shorter than sixteen characters. **What it taught:** “is authentication on?” is not a question a passing test suite answers. The test suite was green while the bypass was live on the main branch.

7.2 One tool returned unfenced text

`read_document_as_markdown` wrapped its output in an explicit untrusted-document fence. `search_notes` did not — it returned raw file lines, so a note containing an instruction arrived at the model as ordinary text. Because `create_note` lets the model write into that same folder, the model could plant text it would later read back as if it were the user’s.

Found by: reading the tool layer against the design, not by any test. **Fixed by:** routing search results through the same fence and the same size cap. **What it taught:** a safety property is only a property if every path implements it. Three of four tools had it; the fourth was a closed loop.

7.3 A permission that existed only at runtime

Clicking a saved note failed with `opener.open_path not allowed`. The Tauri capability granted `opener:default`, which includes `open_url` and `reveal_item_in_dir` but **not** `open_path`. The button had never worked, because the only caller was removed with an earlier feature.

Found by: running the application. **Fixed by:** granting `opener:allow-open-path`, scoped to the notes folder rather than left unscoped. **What it taught:** a capability boundary is enforced at the IPC layer, which no unit test crosses. This is the clearest argument for the end-to-end run in section 6.4.

7.4 Two tests that passed for the wrong reason

The symlink test created its link in a temporary notes folder but requested a path in the real one, so the request failed because the file did not exist. The traversal test used four `..` segments, which resolved to `Documents/etc/passwd` — not the root — so it too failed on a missing file. Both asserted only “Not allowed”, which is also what a missing file returns.

Found by: adding an assertion that the refusal must name *containment*. That assertion, applied to the symlink case, immediately exposed the traversal case. **Fixed by:** building the symlink where the request points, and using enough `..` to clamp at the root. **What it taught:** the number of assertions is not the strength of a test. An assertion on the wrong property is worse than no assertion, because it reports success.

7.5 The pattern

Three of the four defects were invisible to the test suite at the moment they existed. Two were found by reading code, one by running it. The suite found exactly one.

This is not an argument against tests. It is an argument about what tests are for: they confirm that behaviour you already believe is correct stays correct. They do not discover which behaviour is missing, because a missing check produces no failure. The secondary research question is therefore answered: **automated testing finds defects in implemented behaviour; reading and running find defects in absent behaviour.**

---

8. Discussion

8.1 The converter is a utility, not a contribution

Pip's document tool converts PDF, DOCX, PPTX and XLSX to Markdown. That is Microsoft's MarkItDown doing the work; our part is a wrapper that spawns it, validates the path, caps the output and fences the result as untrusted data. Roughly two hundred lines. It is in the product because a student should not need a separate application to read a PDF, not because conversion is a novel problem.

Saying so costs nothing and protects the argument. If the converter were presented as the contribution, the first fair question — *why not just use MarkItDown?* — would have no good answer. It is one of seven tools, and the other six are where the boundary does its work.

The measurement exists because the tool is measurable in isolation, and it produced a useful negative result. Converting a document does not merely fail to reduce its token count — it *increases* it, by 12% on the six-page PDF and 4.5% on the DOCX. Structure costs characters, and characters are tokens.

What reduces cost is sending less. Two pages instead of six costs 686 tokens against 1,844 for pasting the whole document, a 63% reduction. And page-level reading is only possible because the file stayed on the machine: a model that receives a path can ask for a range; a model that receives an upload has already been given everything. The saving therefore belongs to the architecture, not to the converter — which is why section 6.4 reports it as a control condition rather than a feature.

8.2 The boundary is load-bearing

It would be possible to read this project as a security layer bolted onto a document tool. The opposite is closer to the truth. Scoped reading is what makes the system affordable, and scoped reading is only safe if there is a boundary to scope against. The permission layer is not decoration; it is the precondition for the efficiency result.

8.3 What the hands can and cannot do

The model can list a granted folder, read documents in it, search text notes, write notes, and generate two kinds of rendered page. It cannot delete, rename, execute a command, read outside the allowlist, fetch a URL, or write anywhere except one folder. This is deliberately small. A larger write surface would make a better demonstration and a worse system, and the project's claim is specifically that a small boundary is more useful to a student than a large capability.

The same reasoning applies inside the window. The document preview renders Markdown rather than showing a text box, and that rendering is sanitised. It has to be: the main window can call Tauri commands, so a script arriving inside a converted document would be code execution against the application, not merely a dirty page. Images are stripped from the preview for a related reason — a document could otherwise make the window fetch a remote URL and tell whoever wrote it that the file had been opened. A permission boundary that stops at the tool layer and ignores the renderer is not a boundary.

8.4 Delivery was removed on purpose

The application prepares a prompt; it does not send it. An earlier build wrote the prompt directly into the chat input using a page script. That script was tested against a DOM stub and never against a live page, on a third-party site whose markup changes without notice. It was removed rather than risked.

The cost is real: the user presses paste. The benefit is that the demonstrated workflow has no dependency on a vendor's internal structure. This is a scope decision, not an oversight, and it is recorded in section 9 rather than presented as a feature.

9. Limitations

Limitation	Consequence
No user study	The claim that students benefit is reasoned, not measured. The survey instrument exists and was not administered. No usability claim is made anywhere in this report.
One document measured	The efficiency result rests on a single six-page PDF and one DOCX. A ten-document corpus was designed and not completed. Percentages are therefore reported per document, never averaged.
Link 7 unobserved	The approval round trip is tested on both halves and has not been watched end to end.
TOCTOU open	A folder can be swapped between validation and conversion.
macOS only	Windows path handling is untested and no claim is made about it.
Not packaged	The application runs from a checkout. Packaging requires bundling the Python helper, bundling the tool server, bundling the tunnel, and code signing — four phases, the last of which requires a paid developer certificate.
Tier 3 needs the internet	The chat client is cloud-hosted. The local tools are local; the model is not.

---

10. Conclusion

A student on a free tier can have agent-like access to their own files without paying for it, under a boundary they set, can revoke, and can read back afterwards.

That is the claim the evidence supports. H1 was confirmed on a live free account through a public tunnel. H2 was confirmed for eight of nine attack classes, with one race condition documented as open. H3 was rejected, and its replacement — that scoped reading is the lever — was confirmed on the documents measured.

Three things this project did not do, and says so: it did not measure a user, it did not measure a corpus, and it did not deliver the prompt into the chat. Each of those is a limitation of a seven-day build rather than a defect of the design, and each is listed in section 9 with its consequence attached.

The finding worth carrying forward is methodological. Of four defects found, three were invisible to a green test suite; two were found by reading code and one by running the application. For a project about permission boundaries, that is the most transferable lesson: the tests confirm what you remembered to check, and the boundary you forgot to check is the one that matters.

---

11. Future work

1. **Package the application.** Bundle the Python helper and the tool server, add a first-run panel for the tunnel token, sign and notarise. Four phases; the last needs an Apple Developer account.
2. **Administer the survey.** Ten to twenty students, a timed task, before and after. This is the single largest gap between the current evidence and a complete evaluation.
3. **Complete the corpus.** Ten documents across PDF, DOCX, PPTX, XLSX and scanned PDF, with conversion time recorded alongside token counts.
4. **Close the TOCTOU window** by converting inside a sandbox rather than validating a path and trusting it.
5. **Scanned PDFs.** Currently return zero pages; requires OCR.
6. **Windows support**, and a per-tool rate limit.
7. **Restore delivery**, if a stable, documented integration point becomes available — as an option, not a dependency.

---

12. References

1. Anthropic. *Model Context Protocol specification.* modelcontextprotocol.io
2. Microsoft. *MarkItDown: convert documents to Markdown.* github.com/microsoft/markitdown
3. Tauri Contributors. *Tauri 2 documentation.* tauri.app
4. Model Context Protocol. *Authorization specification.* modelcontextprotocol.io
5. OWASP Foundation. *Path Traversal.* owasp.org

Appendix A — Tool inventory

Tool	Reads/writes	Guard
`list_files`	reads a folder listing	allowlist; no argument lists the granted folders
`read_document_as_markdown`	reads a document	allowlist, 24,000-character cap, 180-second timeout, fenced as untrusted
`search_notes`	reads text notes	allowlist, depth 4, 60 matches, fenced, capped
`create_note`	writes `.md`	approval required, notes folder only, filename sanitised
`save_digest`	writes `.html`	approval required, structure-only input, max 8 files
`save_quiz`	writes `.html`	approval required, structure-only input, max 20 questions, answer index validated
`reveal_secret`	none	retained from the connectivity spike; used to prove the tunnel

All writes are confined to a single folder. Every call, allowed or refused, appends one line to an audit log the user can read in the application.

Appendix B — Test inventory

`./test.sh` runs, in order:

- **Rust command layer** — 15 tests covering path handling, allowlist round-trip, converter resolution and settings persistence.
- **Frontend** — TypeScript type check and a production build.
- **Tool layer** — twelve groups of assertions against a live server on a private port with its own state directory: tool registration, allowlist acceptance and refusal, traversal and symlink containment, document conversion and page markers, notes search and fencing, the approval round trip in both directions, the audit log, authentication, folder discovery, and artifact rendering including HTML escaping.

The suite exits non-zero on any failure. It is deliberately not the whole of the evaluation: section 7 explains what it cannot see.